

Audit Report

21 September 2026

fallen-icarus - Cardano Swaps

Disclaimer. This report is a time-boxed security review of the artifacts, commits and file hashes identified herein, performed to the best of TxPipe's knowledge, skill and experience through manual expert code review combined with AI-assisted analysis, with all findings validated by qualified human auditors; it is a snapshot in time, covers only the issues actually identified within the stated scope and assessment period, does not claim to detect every vulnerability, and does not extend to any change made to the code afterwards. This report does not constitute an authorization, certification, endorsement or approval by TxPipe for the reviewed system to be deployed on mainnet, released publicly, or used to handle real assets or user funds, nor does it constitute legal, regulatory, financial or investment advice: that decision, and the acceptance of any residual risk, rests solely with the client, which TxPipe recommends supports with additional independent audits and an open bug bounty programme. This report is addressed to the client, may not be relied upon by third parties, and is subject in full to the terms and conditions set out in Appendix A.

Contents

1 - Summary 4
1.1 - Overview 4
1.2 - Process 4
1.3 - Conclusion 5
1.4 - Audited Files 5
2 - Specification 6
2.1 - Scripts 6
2.2 - UTxOs 8
2.3 - Transactions 9
3 - Findings 22
3.1 - CSW-201 Reference Script field not validated throughout the protocol 23
3.2 - CSW-202 Beacon token names are not injective 24
3.3 - CSW-301 Expirations restricted to 1-minute intervals 25
3.4 - CSW-302 Orders can be created with invalid/unauthorized staking credentials 26
3.5 - CSW-303 Code is not formatted 27
3.6 - CSW-304 Assets with an empty policy ID are treated as ADA regardless of asset name . . 28
4 - Minor issues 29
A Appendix 30
A.1 Terms and Conditions of the Commercial Agreement 30
A.2 Issue Guide 32
A.3 Revisions 33
A.4 About Us 33

1 - Summary

This report provides a thorough audit of Cardano Swaps, a fully peer-to-peer order book settlement protocol for the Cardano blockchain. The document examines the protocol's overall design, including its validators, UTxOs, and transaction diagrams that illustrate the operational processes.

The investigation spanned several potential vulnerabilities, including scenarios where attackers might exploit the validators to steal the funds locked in swap UTxOs, take assets without paying the quoted price, or pollute the on-chain order book. It also included evaluating common vulnerabilities, such as double satisfaction and minting policy vulnerabilities.

The audit is conducted without warranties or guarantees of the quality or security of the code. It's important to note that this report only covers identified issues, and we do not claim to have detected all potential vulnerabilities.

1.1 - Overview

Cardano Swaps is a fully peer-to-peer order book settlement protocol built on the Cardano blockchain. It offers two core primitives: **One-Way Swaps**, which behave as classic limit orders that can only be executed at the quoted price or better, and **Two-Way Swaps**, which can be executed in either direction at two independent prices, allowing market makers to profit from a spread. Orders live on-chain as UTxOs that anyone can discover and execute, without centralized batchers or a shared contract: each user interacts through their own sovereign smart contract instance, retaining full custody of their assets and their delegation rights.

The protocol's behaviour is enforced by four Aiken validators: the one-way and two-way **swap scripts**, spend validators that govern swap UTxOs and validate swap execution, and their companion **beacon scripts**, dual-purpose validators that act as minting policies (creating and closing swaps), and staking scripts (updating swaps). Delegating creation, update, and close logic to the beacon script — which runs only once per transaction — keeps executions cheap and allows many swaps to be composed in a single atomic transaction.

Each open order is a swap UTxO holding the offered asset together with three non-transferable beacon tokens that identify the trading pair and its sides, plus an inline `SwapDatum` specifying the price (or prices), and an optional expiration. Creating, updating, or closing a swap is restricted to the owner via staking credential approval, while executing a swap is a public action: a taker consumes the swap UTxO and must produce a continuing output at the same address that preserves the beacons and datum, respects the quoted price and expiration, and only moves the swapped assets. Partial fills are allowed, and any number of swaps can be chained atomically, enabling multi-hop conversions and composability with arbitrary dApps.

1.2 - Process

Our audit process involved a thorough examination of the Cardano Swaps validators. Areas vulnerable to potential security threats were closely scrutinized, including those where attackers could exploit the validators to steal funds locked in swap UTxOs, take assets without paying the quoted price, contaminate swap UTxOs with dust tokens, or pollute and grief the on-chain order book. This also included evaluating common vulnerabilities, such as double satisfaction and minting policy vulnerabilities.

Findings and feedback from the audit were communicated regularly to the Cardano Swaps team through Telegram. Diagrams illustrating the necessary transaction structure for proper interaction

with the protocol are attached as part of this report. The Cardano Swaps team addressed these issues in an efficient and timely manner, enhancing the overall security of the platform.

1.3 - Conclusion

The report finished with no Critical or Major findings, two resolved Minor findings, one resolved Info finding and three acknowledged Info findings as stated in the findings table. From the acknowledged Info findings, two are related to code quality and design decisions, while the last one (CSW-302) was partially addresses by the project team.

Overall, the protocol's design is sound: no Critical or Major vulnerabilities were identified, the resolved findings strengthened the validators' robustness and cost profile, and the acknowledged ones are conscious trade-offs with no security impact on correctly formed transactions. Additionally, minor issues such as coding style and optimizations were reported through the project's GitHub issue tracker (see Minor issues), further strengthening the protocol's quality and efficiency.

1.4 - Audited Files

Below is a list of all audited files in this report. Any files **not** listed here were **not** audited. The final state of the files for the purposes of this report is considered to be commit `e1ab915709def43692dcde3a69fcfa23313a9bcb`.

Filename
./lib/cardano_swaps/common/types.ak
./lib/cardano_swaps/common/utils.ak
./lib/cardano_swaps/one_way_swap/types.ak
./lib/cardano_swaps/one_way_swap/utils.ak
./lib/cardano_swaps/two_way_swap/types.ak
./lib/cardano_swaps/two_way_swap/utils.ak
./validators/one_way_swap.ak
./validators/two_way_swap.ak

2 - Specification

2.1 - Scripts

The following outlines each validator of the protocol, detailing key aspects such as parameters (if any), purposes, and associated redeemers and datums.

2.1.a - One-Way Swap Script

The swap_script is a spend validator that governs One-Way Swap UTxOs. It enforces staking credential approval for owner operations, validates the beacon execution mode (minting or staking), and ensures fair swap execution when a taker interacts with the swap.

- **Validator**: `swap_script` (one_way_swap.ak)
- **Parameters**: None
- **Purpose**: Spend
  - **Redeemer**:

```
SwapRedeemer {
  SpendWithMint
  SpendWithStake
  Swap
}
```

  - **Datum**:

```
SwapDatum {
  beacon_id: PolicyId
  pair_beacon_name: AssetName
  offer_id: PolicyId
  offer_name: AssetName
  offer_beacon_name: AssetName
  ask_id: PolicyId
  ask_name: AssetName
  ask_beacon_name: AssetName
  swap_price: Rational {
    numerator: Int
    denominator: Int
  }
  prev_input: Option<OutputReference>
  expiration: Option<PosixTime>
}
```

2.1.b - One-Way Beacon Script

The beacon_script is a dual-purpose validator that acts as both a minting policy (for creating and closing swaps) and a staking script (for updating swaps). It validates that beacon outputs go to the correct DApp address and carry a properly configured inline datum.

- **Validator**: `beacon_script` (one_way_swap.ak)
- **Parameters**:`dapp_hash: Hash<Blake2b_224, Script>` - the hash of the companion swap_script. Beacon outputs must go to an address whose payment credential matches this hash.
- **Purpose**: Mint
  - **Redeemer**: `()` - creates or closes swaps by minting or burning beacon tokens.
- **Purpose**: Reward
  - **Redeemer**: `()` - updates swap parameters.
- **Purpose**: Publish
  - **Redeemer**: `()` - registers the beacon script's staking credential on-chain.

2.1.c - Two-Way Swap Script

The swap_script is a spend validator that governs Two-Way Swap UTxOs. It supports bidirectional swaps via `TakeAsset1` and `TakeAsset2` redeemers, using two independent prices. The trading pair is sorted (`asset1 < asset2`) and the validator dynamically assigns the offer/ask roles based on the redeemer.

- **Validator**: `swap_script` (two_way_swap.ak)
- **Parameters**: None
- **Purpose**: Spend
  - **Redeemer**:

```
SwapRedeemer {
  SpendWithMint
  SpendWithStake
  TakeAsset1
  TakeAsset2
}
```

  - **Datum**:

```
SwapDatum {
  beacon_id: PolicyId
  pair_beacon_name: AssetName
  asset1_id: PolicyId
  asset1_name: AssetName
  asset1_beacon_name: AssetName
  asset2_id: PolicyId
  asset2_name: AssetName
  asset2_beacon_name: AssetName
  asset1_price: Rational {
    numerator: Int
    denominator: Int
  }
  asset2_price: Rational {
    numerator: Int
    denominator: Int
  }
  prev_input: Option<OutputReference>
  expiration: Option<PosixTime>
}
```

2.1.d - Two-Way Beacon Script

The beacon_script is a dual-purpose validator that acts as both a minting policy and a staking script. It follows the same architecture as the one-way variant, with the pair beacon enforcing asset1 < asset2.

- **Validator**: `beacon_script` (two_way_swap.ak)
- **Parameters**:`dapp_hash: Hash<Blake2b_224, Script>` - the hash of the companion swap_script. Beacon outputs must go to an address whose payment credential matches this hash.
- **Purpose**: Mint
  - **Redeemer**: `()` - creates or closes swaps by minting or burning beacon tokens.
- **Purpose**: Publish
  - **Redeemer**: `()` - registers the beacon script's staking credential on-chain.
- **Purpose**: Reward
  - **Redeemer**: `()` - updates swap parameters.

2.2 - UTxOs

2.2.a - One-Way Swap UTxOs

There can be multiple One-Way Swap UTxOs, one per active swap order. Each holds the three beacon tokens identifying the trading pair, the offer asset being sold, the accumulated ask asset after swaps occur, and an inline one-way `SwapDatum`.

- Address:
  - Payment: `one_way_swap_script`
  - Staking: Any
- Value:
  - 1 `pair_beacon_token`
  - 1 `offer_beacon_token`
  - 1 `ask_beacon_token`
  - N `offer_asset`
  - M `ask_asset`
  - K lovelace
- Datum: Inline `SwapDatum` (one-way):
  - `beacon_id`: `PolicyId`
  - `pair_beacon_name`: `AssetName`
  - `offer_id`: `PolicyId`
  - `offer_name`: `AssetName`
  - `offer_beacon_name`: `AssetName`
  - `ask_id`: `PolicyId`
  - `ask_name`: `AssetName`
  - `ask_beacon_name`: `AssetName`
  - `swap_price`: `Rational { numerator: Int, denominator: Int }`
  - `prev_input`: `Option<OutputReference>`
  - `expiration`: `Option<PosixTime>`
- RefScript: None

2.2.b - Two-Way Swap UTxOs

There can be multiple Two-Way Swap UTxOs, one per active swap order. Each holds the three beacon tokens identifying the trading pair, any quantity of asset1 and/or asset2, and an inline two-way `SwapDatum`. The trading pair is sorted (`asset1 < asset2`).

- Address:
  - Payment: `two_way_swap_script`
  - Staking: Any
- Value:
  - 1 `pair_beacon_token`
  - 1 `asset1_beacon_token`
  - 1 `asset2_beacon_token`
  - N `asset1`
  - M `asset2`
  - K lovelace
- Datum: Inline `SwapDatum` (two-way):
  - `beacon_id`: `PolicyId`
  - `pair_beacon_name`: `AssetName`
  - `asset1_id`: `PolicyId`

  - asset1_name: AssetName
  - asset1_beacon_name: AssetName
  - asset2_id: PolicyId
  - asset2_name: AssetName
  - asset2_beacon_name: AssetName
  - asset1_price: Rational { numerator: Int, denominator: Int }
  - asset2_price: Rational { numerator: Int, denominator: Int }
  - prev_input: Option<OutputReference>
  - expiration: Option<PosixTime>
- RefScript: None

2.3 - Transactions

The purpose of the transaction diagrams that follow is to outline the key aspects of each operation in the protocol. To provide a clearer presentation, we omit details such as the minimum ADA required in each UTxO, transaction fees, the sources of that ADA, etc.

2.3.a - One-Way Swap

Involved scripts:

- ow_swap_script (spend)
- ow_beacon_script (mint / stake / publish)

Involved Addresses:

- ow_swap_addr = { payment: ow_swap_script, staking: Some(owner_stake) }

Involved Assets:

- pair_beacon_token = (beacon_id, pair_beacon_name)
- offer_beacon_token = (beacon_id, offer_beacon_name)
- ask_beacon_token = (beacon_id, ask_beacon_name)
- offer_asset = (offer_id, offer_name)
- ask_asset = (ask_id, ask_name)

where

```
 pair_beacon_name   = sha2_256(serialise(TradingPair(offer_id, offer_name, ask_id,
ask_name)))
 offer_beacon_name  = sha2_256(serialise(OfferAsset(offer_id, offer_name)))
 ask_beacon_name    = sha2_256(serialise(AskAsset(ask_id, ask_name)))
```

2.3.a.a - Register Beacon Script

The only certificate allowed by the publish purpose is CredentialRegistration. This is done once to allow the script to be executed as a staking script for swap updates.

Involved scripts:

- `ow_beacon_script`
  - Publish
    - Redeemer: `()`

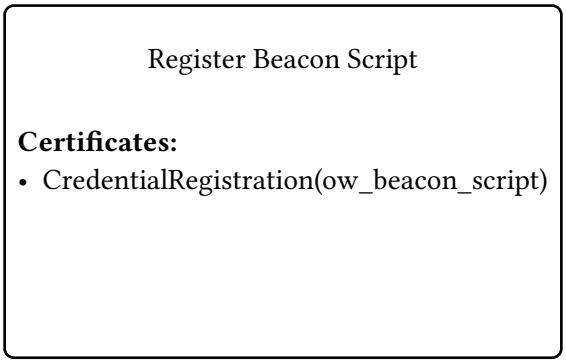
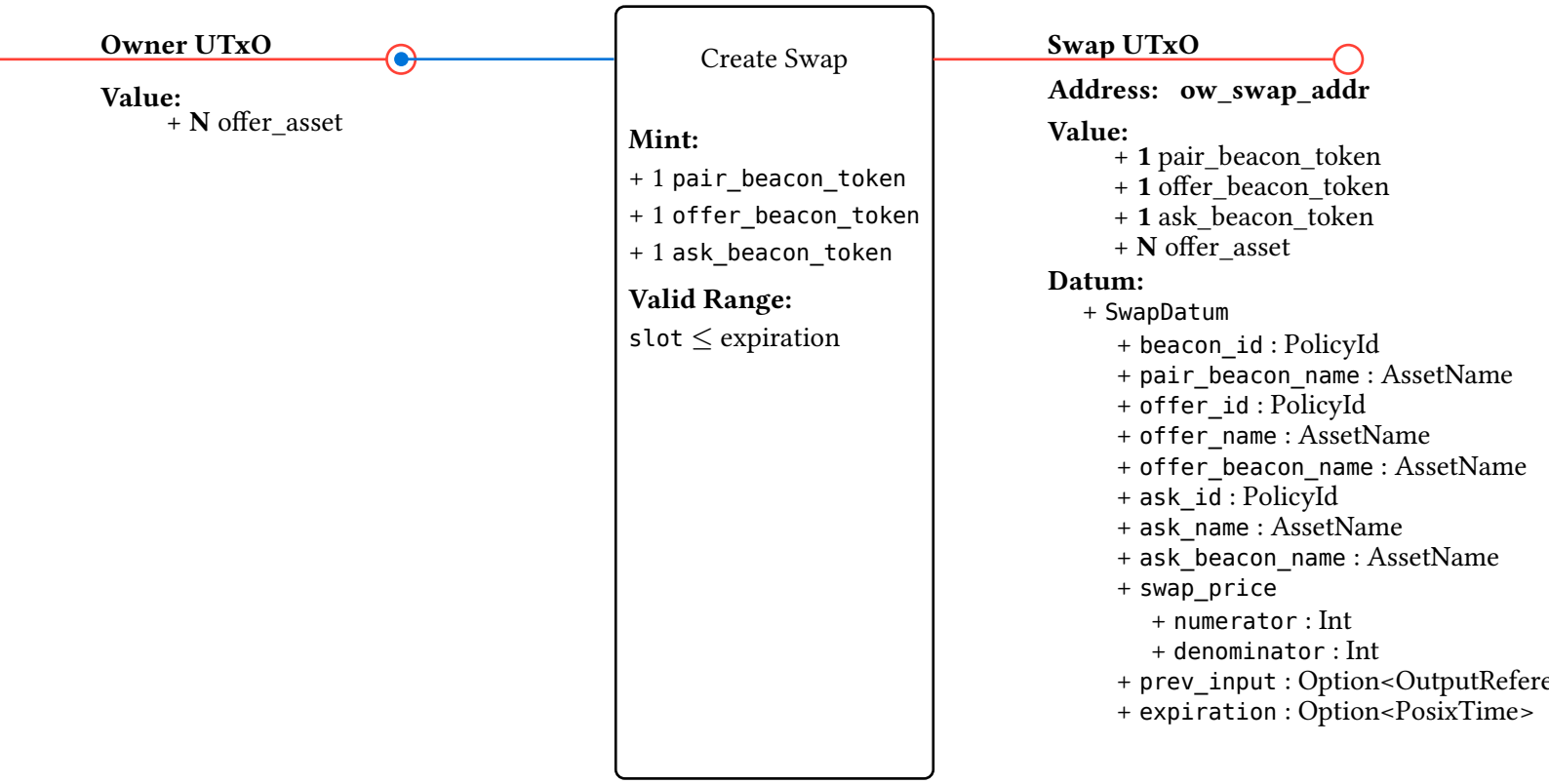


Figure 1: One-Way — Register Beacon Script

2.3.a.b - Create Swap

Involved scripts:

- `ow_beacon_script`
  - ▸ Mint
    - – Redeemer: `()`



Notes:

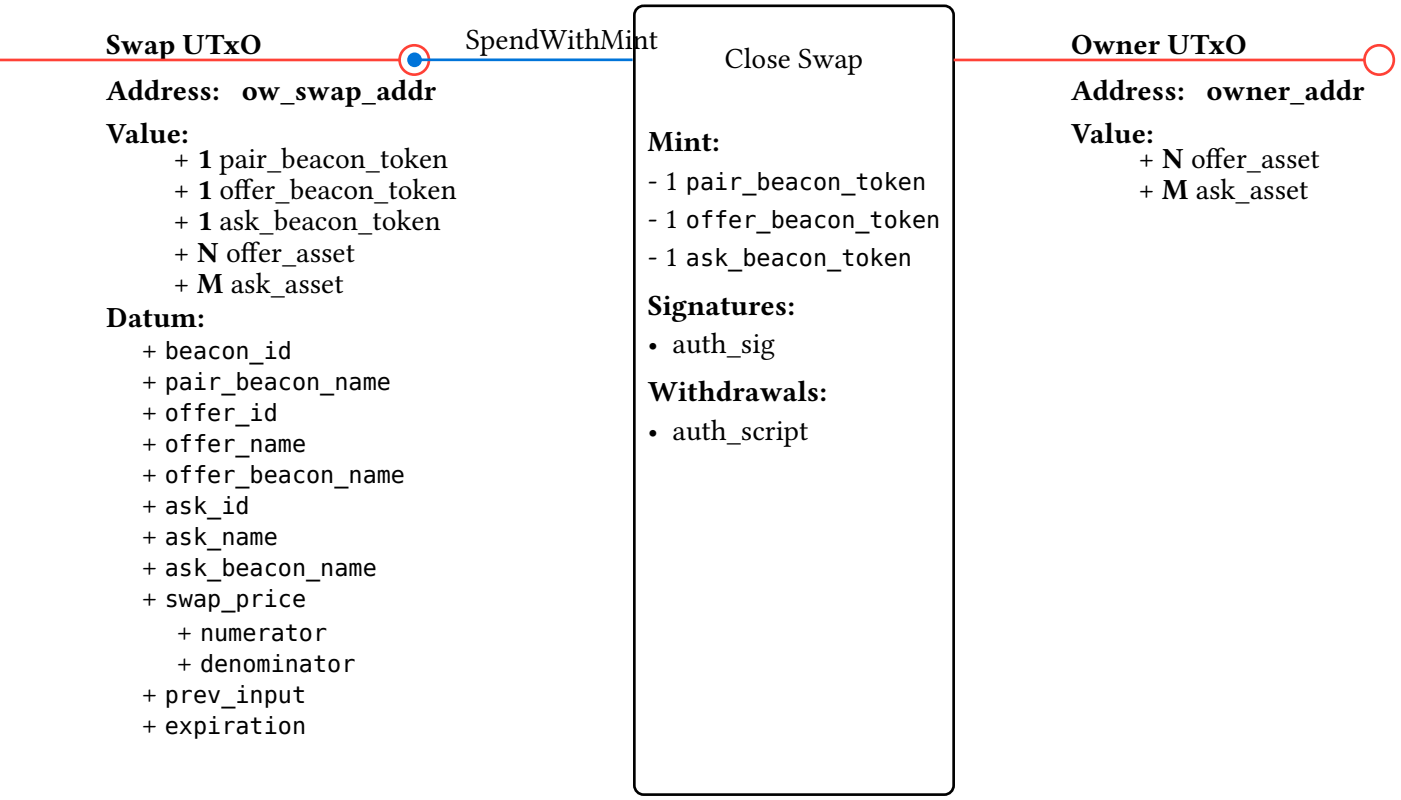
- offer_id ≠ ask_id
- numerator > 0
- denominator > 0
- expiration % 60000 = 0 (when set)

Figure 2: One-Way — Create Swap

2.3.a.c - Close Swap

Involved scripts:

- `ow_swap_script`
  - Spend
    - Redeemer: `SpendWithMint`
- `ow_beacon_script`
  - Mint
    - Redeemer: `()`



Notes:

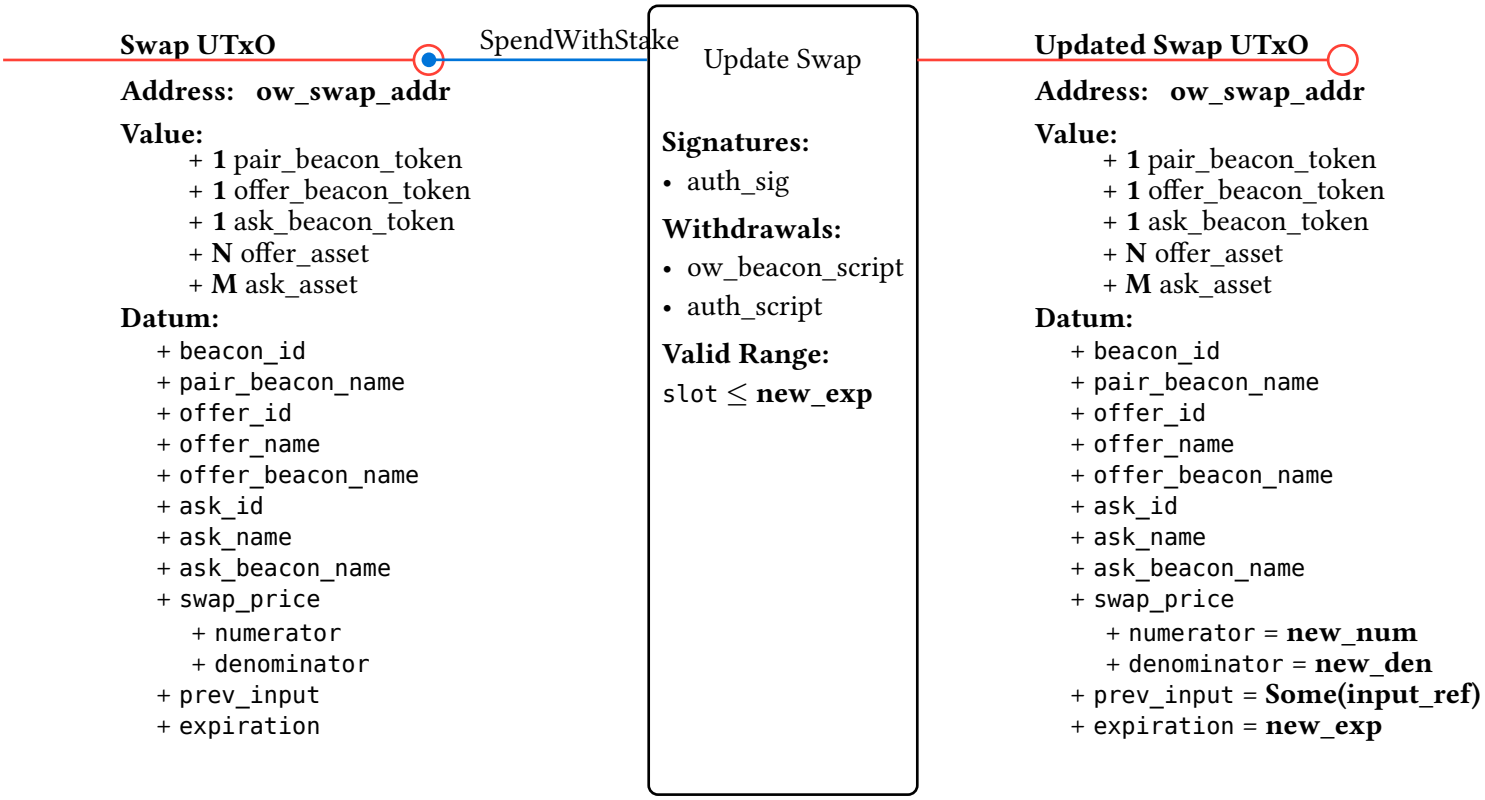
```
(auth_sig, auth_script) =
  when owner_stake is
    VerificationKey(vkey_owner_stake) => (vkey_owner_stake, None)
    Script(script_owner_stake) => (None, script_owner_stake)
```

Figure 3: One-Way — Close Swap

2.3.a.d - Update Swap

Involved scripts:

- `ow_swap_script`
  - Spend
    - Redeemer: `SpendWithStake`
- `ow_beacon_script`
  - Stake
    - Redeemer: `()`



Notes:

```
(auth_sig, auth_script) =
  when owner_stake is
    VerificationKey(vkey_owner_stake) => (vkey_owner_stake, None)
    Script(script_owner_stake) => (None, script_owner_stake)
```

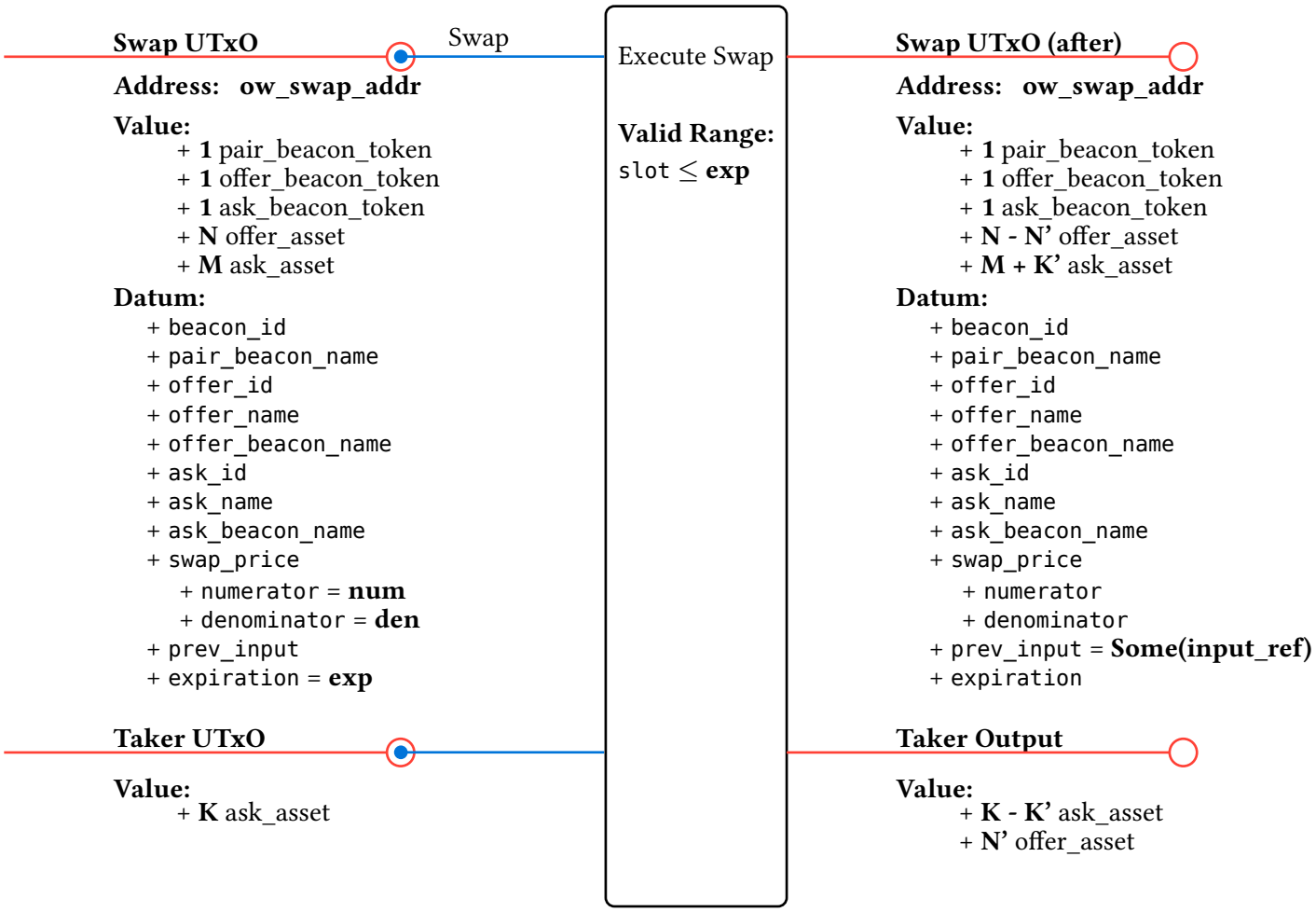
new_num > 0
new_den > 0
input_ref = input ref of Swap UTxO

Figure 4: One-Way — Update Swap

2.3.a.e - Execute Swap

Involved scripts:

- `ow_swap_script`
  - Spend
    - Redeemer: `Swap`



Notes:

- $N' * \text{num} \leq K' * \text{den}$
- $N' \geq 0$
- $K' \geq 0$

Figure 5: One-Way — Execute Swap

2.3.b - Two-Way Swap

Involved scripts:

- `tw_swap_script` (spend)
- `tw_beacon_script` (mint / stake / publish)

Involved Addresses:

- `tw_swap_addr = { payment: tw_swap_script, staking: Some(owner_stake) }`

Involved Assets:

- `pair_beacon_token   = (beacon_id, pair_beacon_name)`
- `asset1_beacon_token = (beacon_id, asset1_beacon_name)`
- `asset2_beacon_token = (beacon_id, asset2_beacon_name)`
- `asset1              = (asset1_id, asset1_name)`
- `asset2              = (asset2_id, asset2_name)`

where

```
 pair_beacon_name    = sha2_256(serialise(SortedPair(asset1_id, asset1_name,
asset2_id, asset2_name)))
 asset1_beacon_name  = sha2_256(serialise(Asset(asset1_id, asset1_name)))
 asset2_beacon_name  = sha2_256(serialise(Asset(asset2_id, asset2_name)))
```

2.3.b.a - Register Beacon Script

The only certificate allowed by the publish purpose is CredentialRegistration. This is done once to allow the script to be executed as a staking script for swap updates.

Involved scripts:

- `tw_beacon_script`
  - Publish
    - Redeemer: `()`

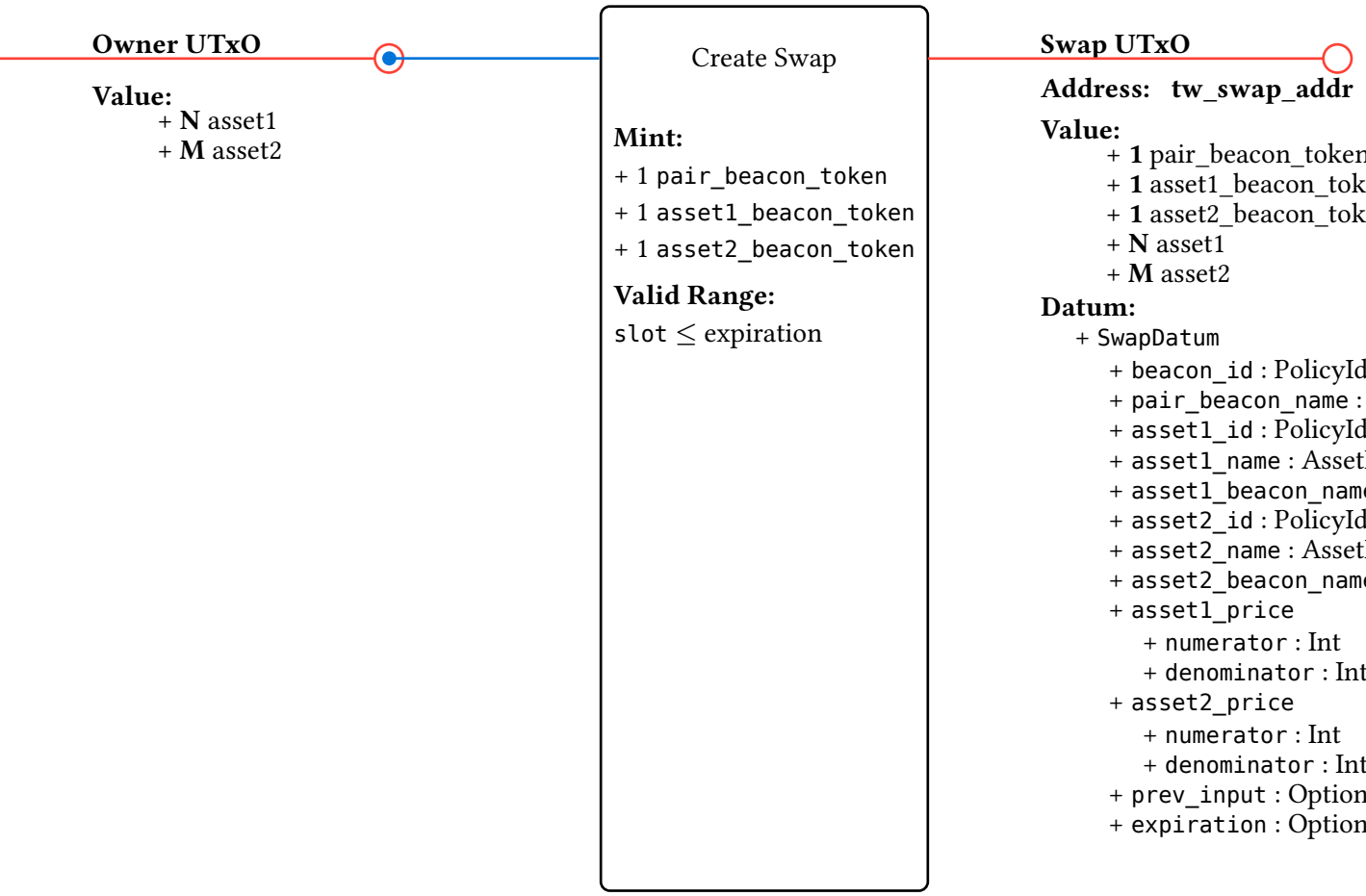


Figure 6: Two-Way — Register Beacon Script

2.3.b.b - Create Swap

Involved scripts:

- `tw_beacon_script`
  - Mint
    - Redeemer: `()`



Notes:

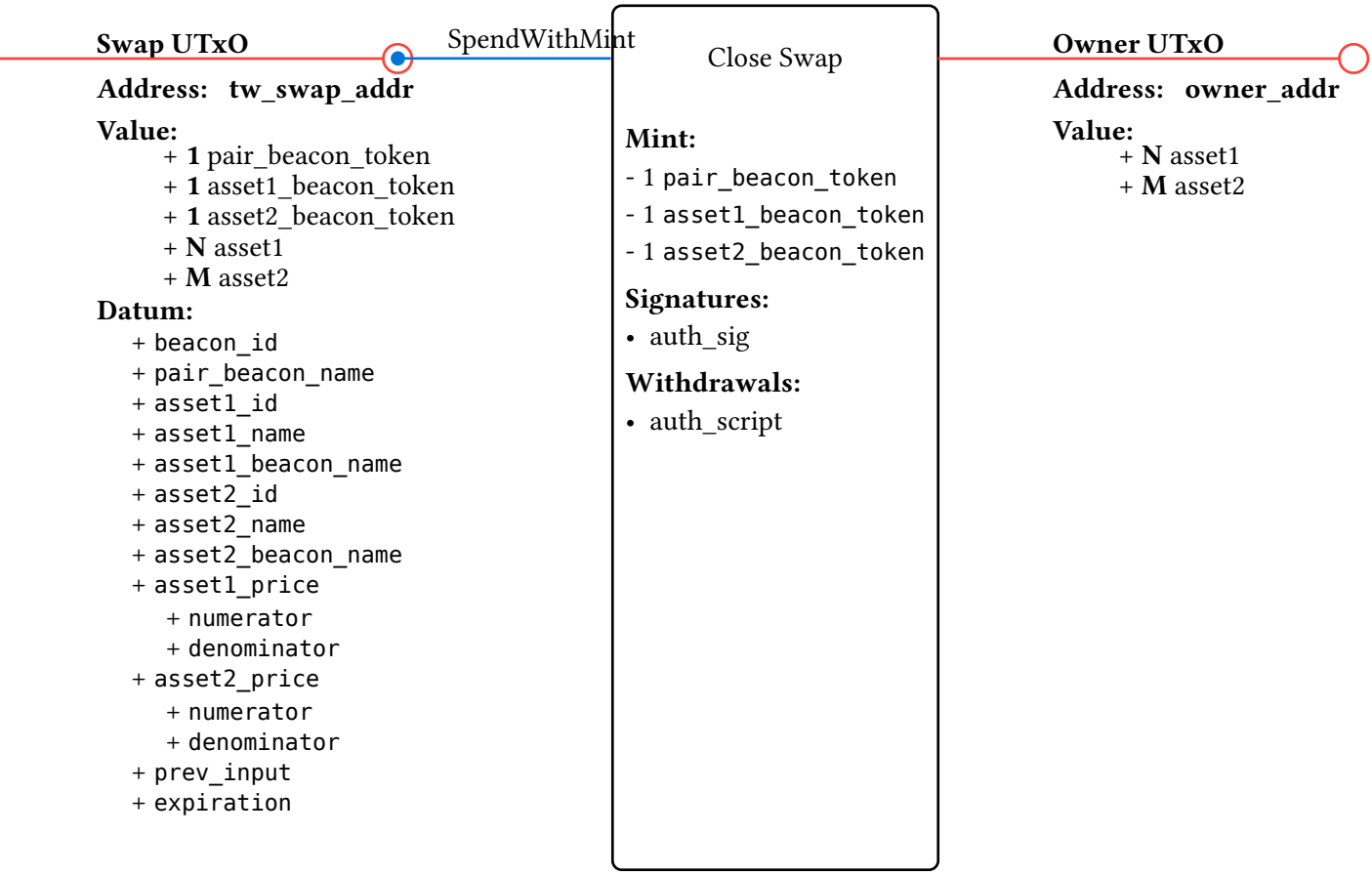
- asset1 < asset2
- numerator > 0
- denominator > 0
- expiration % 60000 = 0 (when set)

Figure 7: Two-Way — Create Swap

2.3.b.c - Close Swap

Involved scripts:

- `tw_swap_script`
  - Spend
    - Redeemer: `SpendWithMint`
- `tw_beacon_script`
  - Mint
    - Redeemer: `()`



Notes:

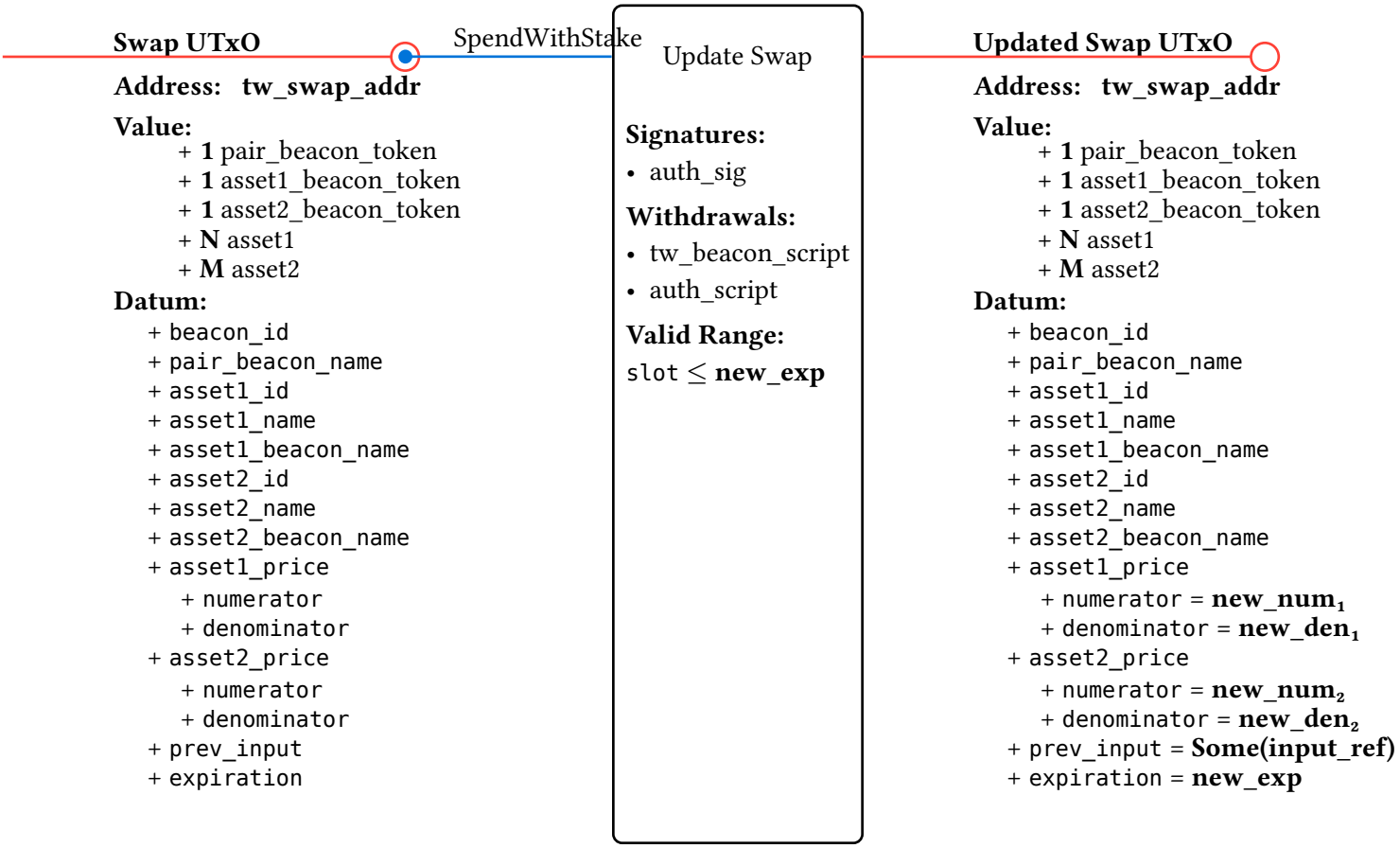
```
(auth_sig, auth_script) =
  when owner_stake is
    VerificationKey(vkey_owner_stake) => (vkey_owner_stake, None)
    Script(script_owner_stake) => (None, script_owner_stake)
```

Figure 8: Two-Way — Close Swap

2.3.b.d - Update Swap

Involved scripts:

- `tw_swap_script`
  - Spend
    - Redeemer: `SpendWithStake`
- `tw_beacon_script`
  - Stake
    - Redeemer: `()`



Notes:

```
(auth_sig, auth_script) =
  when owner_stake is
    VerificationKey(vkey_owner_stake) => (vkey_owner_stake, None)
    Script(script_owner_stake) => (None, script_owner_stake)
```

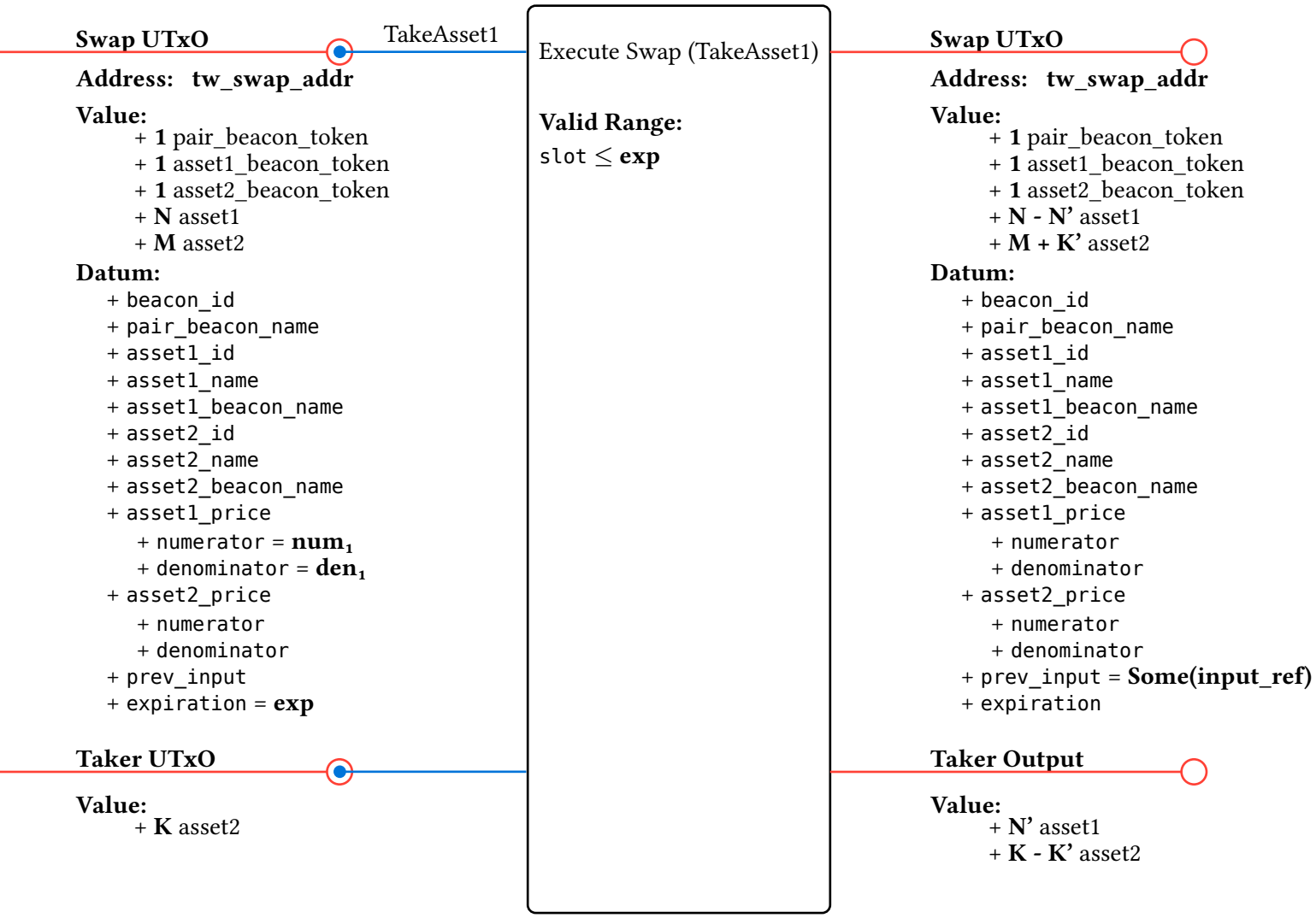
$\text{new_num}_1 > 0$
$\text{new_den}_1 > 0$
$\text{new_num}_2 > 0$
$\text{new_den}_2 > 0$
input_ref = input ref of Swap UTxO

Figure 9: Two-Way — Update Swap

2.3.b.e - Execute Swap — TakeAsset1

Involved scripts:

- `tw_swap_script`
  - ▸ Spend
    - – Redeemer: `TakeAsset1`



Notes:

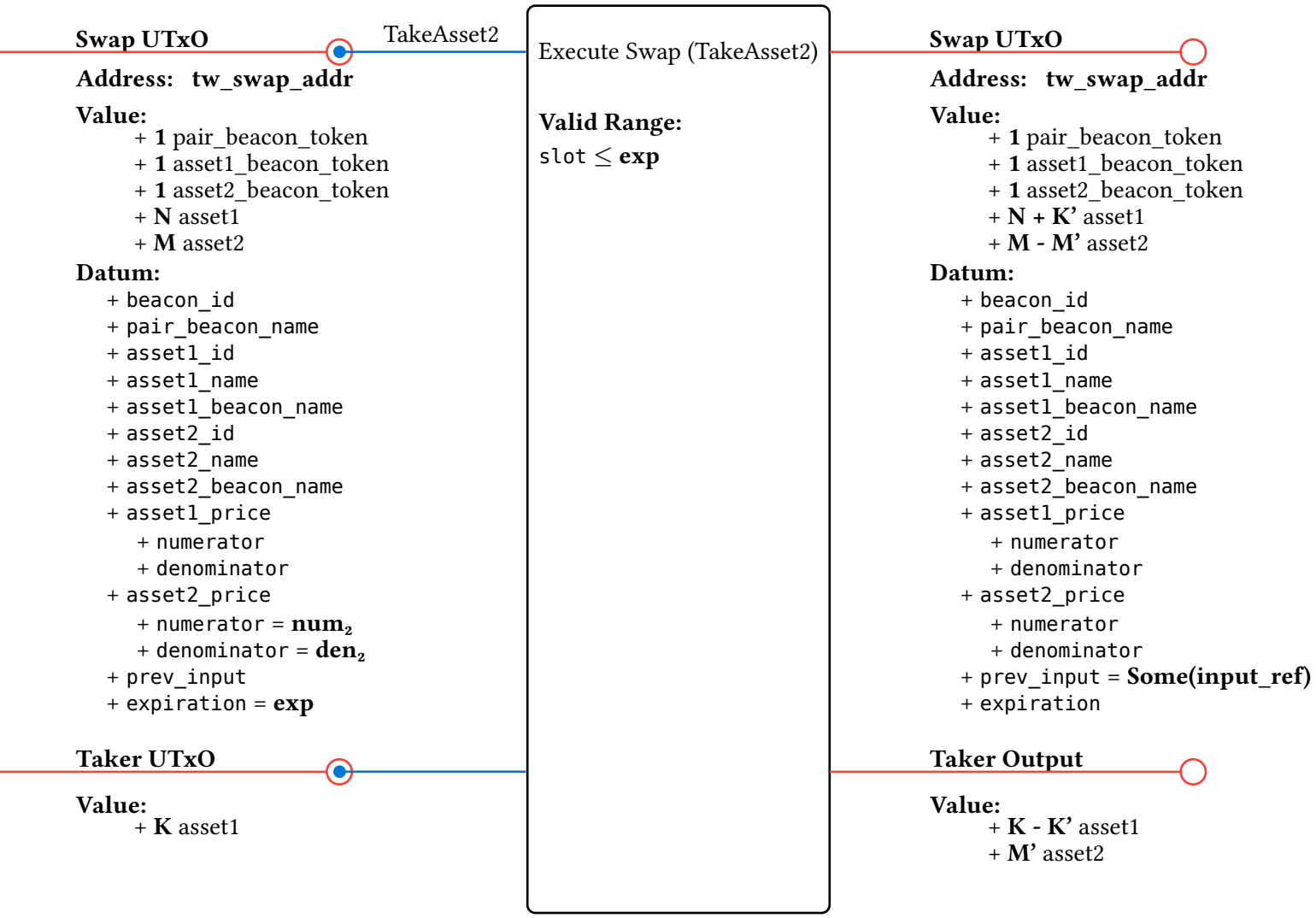
- $N' * \text{num}_1 \leq K' * \text{den}_1$
- $N' \geq 0$
- $K' \geq 0$

Figure 10: Two-Way — Execute Swap (TakeAsset1)

2.3.b.f - Execute Swap — TakeAsset2

Involved scripts:

- `tw_swap_script`
  - ▸ Spend
    - – Redeemer: `TakeAsset2`



Notes:

- $M' * \text{num}_2 \leq K' * \text{den}_2$
- $M' \geq 0$
- $K' \geq 0$

Figure 11: Two-Way — Execute Swap (TakeAsset2)

3 - Findings

ID	Title	Severity	Status
CSW-201	Reference Script field not validated throughout the protocol	Minor	Resolved
CSW-202	Beacon token names are not injective	Minor	Resolved
CSW-301	Expirations restricted to 1-minute intervals	Info	Acknowledged
CSW-302	Orders can be created with invalid/unauthorized staking credentials	Info	Acknowledged
CSW-303	Code is not formatted	Info	Acknowledged
CSW-304	Assets with an empty policy ID are treated as ADA regardless of asset name	Info	Resolved

3.1 - CSW-201 Reference Script field not validated throughout the protocol

Category	Commit	Severity	Status
Bug	c40528cd53889baf560307827168b3d99f5f9b7b	Minor	Resolved

3.1.a - Description

With the addition of the `minFeeRefScriptsCoinsPerByte` protocol parameter, including a reference script in any input will impact the transaction fees, regardless of whether the script is executed.

With this in mind, there is an attack vector where a malicious party includes a huge reference script in outputs of a transaction. This ends up costing more fees to the next party interacting with those UTxOs.

In particular, someone executing a swap can include a reference script in the output of the transaction, costing up to 0.24 ADAs in additional fees to the next consumer. For a protocol that has financial efficiency as a core pillar, and given that the memory and cpu cost of a swap stand at around 0.02 ADAs, this 10x increase can make an order significantly less attractive.

3.1.b - Recommendation

We suggest checking that the reference script field of the swap output is None

3.1.c - Resolution

Resolved in commit `8670e19e922c68c6f5ceb6dd49ef52830df3c165`

3.2 - CSW-202 Beacon token names are not injective

Category	Commit	Severity	Status
Bug	d952f7e4c1391ec6de556fa071089b0358a40c4b	Minor	Resolved

3.2.a - Description

Beacon token names for trading pairs are computed by hashing the raw concatenation of the pair's asset policy ids and asset names (`sha2_256(asset1_id ++ asset1_name ++ asset2_id ++ asset2_name)`), without domain separation or length checking.

Given that bytestring lengths are not enforced, the field boundaries are ambiguous, so the same byte string can be parsed as two different asset pairs. Two pairs with the same first policy, e.g. `(id1, n1, id2, n2)` and `(id1, n1 ++ id2[0..k], id2[k..28] ++ n2[0..k], n2[k..])`, both concatenate to `id1 ++ n1 ++ id2 ++ n2` and therefore hash to the same pair beacon, even though the second asset is a different token.

The two-way swap's asset beacon allows a cheaper alias: the pair beacon of `{(P, ""), (Q, "")}` is `sha2_256(P ++ Q)`, which equals the asset beacon of any token under policy `P` whose 28-byte name equals `Q`, so every order for that token and every order for the pair share the same beacon name.

Because two distinct asset pairs can therefore produce the same preimage, an attacker can list an order for a different pair that shares the victim pair's beacon, polluting the victim pair's order book queries. Furthermore, this breaks the invariant that every beacon token name is unique for the combination it represents and can result in attacks or vulnerabilities in the future.

3.2.b - Recommendation

We suggest replacing the `bytearray.concat` preimages with a dedicated data type that gets encoded using Aiken's serialise function before hashing. The best approach would be having distinct constructors per beacon type (`Pair(asset1, asset2)`, `Offer(asset)`, `Ask(asset)`, `Asset(asset)`). The CBOR encoding adds delimiters and length prefixes between every field, making the encoding injective and eliminating every constructible collision, while the distinct constructor tags provide the domain separation between pair, offer, ask, and asset namespaces, removing the need for the `#"01"`/`#"02"`/`#"00"` constants.

3.2.c - Resolution

Resolved in commit `381be35602629de1866f708ef156801d7d04f69b`

3.3 - CSW-301 Expirations restricted to 1-minute intervals

Category	Commit	Severity	Status
Design	5657d09c2f85dec74b28f57ec6e71cfd283620e9	Info	Acknowledged

3.3.a - Description

In both the one-way and two-way `SwapDatum`, the `expiration` field is restricted by the on-chain code to fall on 1-minute intervals. When an expiration is set, `beacon_destination_check` requires `expiration >= invalid-hereafter` and `expiration % 60000 == 0`. This is justified in the code with: `This way front-ends will not struggle to keep up with the on-chain order book.` While this doesn't constitute as an attack vector or vulnerability, in general, it is recommended to avoid on-chain checks that exist solely to accommodate potential off-chain or front-end limitations.

3.3.b - Recommendation

We suggest removing this check to improve script size and cost.

3.3.c - Resolution

The **project team** decided not to resolve this finding, keeping the 1-minute expiration intervals as a deliberate design choice, citing the following reasons:

- It serves as a system-wide invariant for light node implementations
- It standardizes how often clients need to prune expired orders that are still on-chain
- An off-chain floor would not help, since orders expiring between blocks, staggered by milliseconds, would still force constant client updates
- The on-chain cost is not significant

As the **audit team**, we endorse the decision, as this finding is only informational and there's no security impact.

3.4 - CSW-302 Orders can be created with invalid/unauthorized staking credentials

Category	Commit	Severity	Status
Robustness	d952f7e4c1391ec6de556fa071089b0358a40c4b	Info	Acknowledged

3.4.a - Description

The protocol specification classifies swap creation as an owner action: *"Creating, updating, or closing a swap is restricted to the owner (via staking credential approval)"* and states that minted beacons are sent to an address which must have a *valid staking credential*.

On-chain, however, the mint handler performs almost no validation. It only validates that newly minted beacons are sent to an address with **some** stake credential. This is in contrast to the staking_credential_approves checks that the close or update spending paths execute.

Therefore, any party may mint beacons and create swap UTxOs at any valid DApp address, including addresses whose stake credential they do not control, or even worse, don't exist. In the latter case, the order continues to operate as normal, and users can trade against it, but the funds become irrecoverable as the order can't be updated or closed.

This behaviour can be compared to creating an order with intentionally bad datum fields, which fall under user responsibility. The main difference is that here beacon tokens get minted and locked at these unrecoverable orders, while bad datum orders require creation outside of the protocol and won't have official beacon tokens.

3.4.b - Recommendation

We recommend updating the minting validation to run staking_credential_approves, to make sure that the credential can be executed and funds won't be lost.

3.4.c - Resolution

The **project team** partially addresses this finding in commit `238955780670e971bed099fd45e7c9c364363c24`. Because the minting policy checks against all outputs that have beacon tokens, differentiation between newly created and swapped against orders becomes difficult. In order to not lose composability with the swap operation, the team implemented the authorization check against orders that have the `pev_input` field set to `None`. This is a partial fix because, while swapped orders are forced to have `Some` in that field, newly created orders can have any value.

This means that it is still possible to create orders with invalid staking credentials by creating the order with Some output reference in the `prev_input` field.

As the **audit team** we acknowledge this partial resolution as a step in a good direction, covering the most common case and making the mistake less probale. It can still be mitigated by inspecting the tx carefully and ensuring new orders have `prev_input` set to `None`

3.5 - CSW-303 Code is not formatted

Category	Commit	Severity	Status
Quality	45c393ad8e38e3dec14431396000d6fa3db15436	Info	Acknowledged

3.5.a - Description

The current code is not formatted with the standard `aiken fmt` formatting tool. This results in inconsistent style across modules (indentation, spacing, line breaks), which makes the code harder to read and review, adds noise to diffs, and can obscure actual logic changes behind purely cosmetic ones.

3.5.b - Recommendation

We recommend running `aiken fmt` on the codebase to ensure consistent formatting.

3.5.c - Resolution

The **project team** decided not to run the formatter citing differences in preference with some formatter rules, as well as an intention to maintain the code changes to a minimum to avoid clogging the changelog. They also noted their intention to consider the use of the formatter in future versions, when the codebase would undergo more exhaustive code refactoring either way. As the **audit team** we acknowledge this and understand the decision, as this is a purely cosmetic finding and there's no security or performance impact.

3.6 - CSW-304 Assets with an empty policy ID are treated as ADA regardless of asset name

Category	Commit	Severity	Status
Robustness	45c393ad8e38e3dec14431396000d6fa3db15436	Info	Resolved

3.6.a - Description

In both swap implementations, `extract_ask_and_offer_quantity` identifies ADA using only the policy ID. If an offer or ask is defined with an empty policy ID and a non-empty asset name, the function assigns the real lovelace quantity to that asset, even though such an asset cannot exist in an on-chain value.

This allows malformed orders with valid beacons to account for and exchange lovelace in place of the asset declared in the datum, creating an inconsistency between the declared and accepted assets. This also means that orders that deal with ADAs could have different beacon tokens. The normal swap price constraints remain enforced, and correctly formed orders are not affected. ADAs get counted as offered assets first, so if both the offer and ask assets have an empty policy, the ask quantity will always be 0 and the order becomes unfillable.

3.6.b - Recommendation

If the team wishes to patch this behaviour, we recommend checking that the `offer_name` or `ask_name` equals “” in the ADA branch of `extract_ask_and_offer_quantity`.

3.6.c - Resolution

Resolved in commit `fe9d064de25512804e7d2471e31663c782dfe63b`

4 - Minor issues

In this section we list some issues we found that do not qualify as findings such as coding style and optimizations. We used the Github issues system to report them.

- #28 `has_beacon_script_minting_execution` can be optimized
- #29 `extract_ask_and_offer_quantity` can be optimized
- #32 Reduce duplicated code
- #33 Collapse nested ifs into and expression
- #35 Move beacon check outside fold in `extract_ask_and_offer_quantity`

A Appendix

A.1 Terms and Conditions of the Commercial Agreement

A.1.1 Confidentiality

Both parties agree, within a framework of trust, to discretion and confidentiality in handling the business. The Client may share, distribute and publish this report freely, provided that it is reproduced in full and without alteration, including this Appendix. Any partial reproduction, excerpt, quotation or summary of this report requires the prior written consent of TxPipe LLC, 651 N Broad St, Suite 201, Middletown, registered in the county of New Castle. Such sharing or publication grants no third party any right to rely on this report, and TxPipe assumes no duty of care or liability towards any person other than the Client. Neither the Client nor any third party may use TxPipe's name or logo for marketing or promotional purposes, or present this report as a certification, endorsement or guarantee of security, without TxPipe's prior express written authorization; a factual statement that TxPipe performed the audit, accompanied by a link or reference to the full report, is permitted.

The violation of the aforementioned, as stated supra, shall empower TxPipe to pursue all of its rights and claims in accordance with the provisions outlined in Title 6, Subtitle 2, Chapter 20 of the Delaware Code titled "Trade Secrets,", and to also invoke any other applicable law that protects or upholds these rights.

Therefore, in the event of any harm inflicted upon the company's reputation or resulting from the misappropriation of trade secrets, the company hereby reserves the right to initiate legal action against the contractor for the actual losses incurred due to misappropriation, as well as for any unjust enrichment resulting from misappropriation that has not been accounted for in the calculation of actual losses.

A.1.2 Service Extension and Details

This report does not endorse or disapprove any specific project, team, code, technology, asset or similar. It provides no warranty or guarantee about the quality or nature of the technology/code analyzed.

This agreement does not authorize the client fallen-icarus to make use of the logo, name, or any other unauthorized reference to Txpipe LLC, except upon express authorization from the company.

TxPipe LLC shall not be liable for any use or damages suffered by the client or third-party agents, nor for any damages caused by them to third parties. The sole purpose of this commercial agreement is the delivery of what has been agreed upon. The company shall be exempt from any matters not expressly covered within the contract, with the client bearing sole responsibility for any uses or damages that may arise.

Any claims against the company under the aforementioned terms shall be dismissed, and the client may be held accountable for damages to reputation or costs resulting from non-compliance with the aforementioned provisions. **This report provides general information and is not intended to constitute financial, investment, tax, legal, regulatory, or any other form of advice.**

Any conflict or controversy arising under this commercial agreement or subsequent agreements shall be resolved in good faith between the parties. If such negotiations do not result in a conventional agreement, the parties agree to submit disputes to the courts of Delaware and to the laws of that jurisdiction under the powers conferred by the Delaware Code, TITLE 6, SUBTITLE I, ARTICLE 1, Part 3 § 1-301. and Title 6, SUBTITLE II, chapter 27 §2708.

A.1.3 Disclaimer

The audit constitutes a comprehensive examination and assessment as of the date of report submission. The company expressly disclaims any certification or endorsement regarding the subsequent performance, effectiveness, or efficiency of the contracted entity, post-report delivery, whether resulting from modification, alteration, malfeasance, or negligence by any third party external to the company.

The company explicitly disclaims any responsibility for reviewing or certifying transactions occurring between the client and third parties, including the purchase or sale of products and services.

This report is strictly provided for ***informational purposes*** and reflects solely the due diligence conducted on the following files and their corresponding hashes using sha256 algorithm:

Filename: `./lib/cardano_swaps/common/types.ak` **Hash**: `90d72a486eac68d92b5229a441228e1edd76e74174a3a186aa5aa327e6a48ee0`
Filename: `./lib/cardano_swaps/common/utils.ak` **Hash**: `da57cb57b01f6479c059a32bddc298b5b10fff9462f1ba8b12adf6372c70439f`
Filename: `./lib/cardano_swaps/one_way_swap/types.ak` **Hash**: `195a49aa463ea34a3db8b47acd197fc4dcb77fe1ea3ff0cdce43197d97e2b0ea`
Filename: `./lib/cardano_swaps/one_way_swap/utils.ak` **Hash**: `7adce48facdec1719e1c1c42ca5e07b5e727ea4c6e6c012ee442a13e1583f74c`
Filename: `./lib/cardano_swaps/two_way_swap/types.ak` **Hash**: `7deaa3bb842077f03e96732f78c008cc08bf58dd04eec6b40f10766f656cdb30`
Filename: `./lib/cardano_swaps/two_way_swap/utils.ak` **Hash**: `f41f2a0fd7b9256b2fbb2dc53eeedb4ce1c1d5349e547b400d25d0600d89dd16`
Filename: `./validators/one_way_swap.ak` **Hash**: `2010a2cf14b34b093a619c00f3f507bcf28b07b42be6d17d5d63454fe8cb8d45`
Filename: `./validators/two_way_swap.ak` **Hash**: `260ab6ae4bbf970f1d30273c4b6e22fabe4a30184ba803a13a2033a58c7b762e`

TxPipe advocates for the implementation of multiple independent audits, a publicly accessible bug bounty program, and continuous security auditing and monitoring. Despite the diligent manual review processes, the potential for errors exists. TxPipe strongly advises seeking multiple independent opinions on critical matters. It is the firm belief of TxPipe that every entity and individual is responsible for conducting their own due diligence and maintaining ongoing security measures.

A.2 Issue Guide

A.2.1 Severity

Severity	Description
Critical	Critical issues highlight exploits, bugs, loss of funds, or other vulnerabilities that prevent the dApp from working as intended. These issues have no workaround.
Major	Major issues highlight exploits, bugs, or other vulnerabilities that cause unexpected transaction failures or may be used to trick general users of the dApp. dApps with Major issues may still be functional.
Minor	Minor issues highlight edge cases where a user can purposefully use the dApp in a non-incentivized way and often lead to a disadvantage for the user.
Info	Info are not issues. These are just pieces of information that are beneficial to the dApp creator. These are not necessarily acted on or have a resolution, they are logged for the completeness of the audit.

A.2.2 Status

Status	Description
Resolved	Issues that have been **fixed** by the **project** team.
Acknowledged	Issues that have been **acknowledged** or **partially fixed** by the **project** team. Projects can decide to not **fix** issues for whatever reason.
Identified	Issues that have been **identified** by the **audit** team. These are waiting for a response from the **project** team.

A.3 Revisions

This report was created using a git based workflow. All changes are tracked in a github repo and the report is produced using typst. The report source is available here. All versions with downloadable PDFs can be found on the releases page.

A.4 About Us

TxPipe is a blockchain technology company responsible for many projects that are now a critical part of the Cardano ecosystem. Our team built Oura, Scrolls, Pallas, Demeter, and we're the original home of Aiken. We're passionate about making tools that make it easier to build on Cardano. We believe that blockchain adoption can be accelerated by improving developer experience. We develop blockchain tools, leveraging the open-source community and its methodologies.

A.4.1 Links

- Website
- Email
- Twitter